

Conjugate Operators for Transparent, Explorable Research Outputs

ANONYMOUS AUTHOR(S)

Charts, figures, and text derived from data play an important role in decision making, from data-driven policy development to day-to-day choices informed by online articles. Making sense of, or fact-checking, outputs means understanding how they relate to the underlying data. Even for domain experts with access to the source code and data sets, this poses a significant challenge. In this paper we introduce a new program analysis framework which supports interactive exploration of fine-grained I/O relationships directly through computed outputs, making use of dynamic dependence graphs. This framework enables a novel notion in data provenance which we call *linked inputs*, a relation of mutual relevance or “cognacy” which arises between inputs when they contribute to common features of the output. Queries of this form allow readers to ask questions like “What outputs use this data element, and what other data elements are used along with it?”. We show how Jonsson and Tarski’s concept of conjugate operators on Boolean algebras appropriately characterises the notion of cognacy in a dependence graph, and give a procedure for computing linked inputs over such a graph.

To demonstrate the approach in practice, we present a functional programming language called [OurLanguage] which automatically enriches visual outputs with interactions supporting linked inputs and similar fine-grained queries, and argue how these can aid comprehension using realistic examples. We show how to obtain more informative, contextualised queries via projection operators which factor out data contributed by specific inputs or demand associated with specific outputs. We also show how to derive a *linked outputs* operator, capturing a dual cognacy relation between output features whenever they compete for common input data. Composed with suitable projections, this recovers a prior approach to linked outputs based on Galois connections and execution traces. As well as generalising and extending this prior work, the graph approach presented in this paper performs better (computationally) on most examples and is significantly simpler to implement, and thus easier to deploy to other languages.

1 INTRODUCTION: TOWARDS TRANSPARENT RESEARCH OUTPUTS

Whether formulating a national policy or simply deciding what groceries to buy, we increasingly rely on the charts, figures and text created by scientists and journalists. Interpreting these visual and textual summaries is essential to making informed decisions. However, most of the artefacts we encounter are *opaque*: they are unable to reveal anything about how they relate to the data they were derived from. Whilst one could in principle try to use the source code and data sources to reverse engineer some of these relationships, this requires substantial expertise, as well as valuable time spent away from the “comprehension context” in which we encountered the output in question. These difficulties are only compounded when the information presented draws on multiple data sources, such as medical meta-analyses [DerSimonian and Laird 1986], ensemble models in climate science [Murphy et al. 2004], or queries that span multiple database tables [Selinger et al. 1979]. Even professional reviewers may lack the resources or inclination to embark on such an activity. Perhaps more often than we would like, we end up taking things on trust.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2024 Association for Computing Machinery.

2475-1421/2024/7-ART1 \$15.00

<https://doi.org/>

With traditional print media, a “disconnect” between outputs and underlying data is unavoidable. For digital media, other options are open to us. One way to improve things is to engineer visual artefacts to be more “self-explanatory” so they can reveal the relationship to the underlying data to an interested user. Consider the histogram in Figure 1 showing urban population growth in Asia over a 10-year period [Bremer and Ranzijn 2015]. There are many questions a reader might have about what this chart represents — in other words, how visual elements map to underlying data. Do the points represent individual cities? What does the colour scheme indicate? Which of the points represent large cities or small cities? Legends and other accompanying text can help, but some ambiguities inevitably remain. The entire value proposition of a summary after all is that it presents the big picture at the expense of certain detail.

Bremer and Ranzijn implemented an interactive feature that allow a user to explore some of these questions themselves *in situ*, i.e., directly from the chart. By selecting an individual point, they are able to bring up a view showing the data that point was calculated from. In this case the highlighted dot represents data for Chiang Mai and the plotted value of 10.9% was derived from an increase in population density from 4,416 to 5,010 people per sq. km.

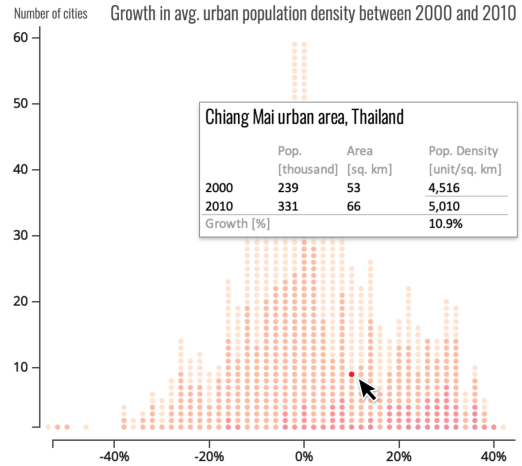


Fig. 1. Hand-crafted data-transparent visualisation [Bremer and Ranzijn 2015]

1.1 Data transparency as PL infrastructure

Features like these are valuable comprehension aids but are laborious to implement by hand, as done by Bremer and Ranzijn [2015]. Manual implementations also require the author to anticipate the queries a user might have and do not always generalise well: for example Bremer and Ranzijn’s visualisation only allows the user to select one point at a time.

For these reasons, there has been recent interest in treating data exploration as a programming language infrastructure problem, baking dependency metadata directly into outputs so that interactive queries of this sort can be supported automatically [Ricciotti et al. 2017; Perera et al. 2022]. The author of the content no longer has to concern themselves with these tricky features, and in fact may find that they have an easier time exploring data, and building more complex models. Similarly, the consumer of the content now has the ability to explore detail on an as-needed basis in the context of the high-level perspective offered by the visual summary. In contrast to Bremer and Ranzijn [2015], the relationship to data can be incrementally discoverable, revealed as the user interacts with the parts of the output they care about, ideally with some kind of formal guarantee that the revealed data is in some sense “minimal and sufficient”.

We follow the direction of Ricciotti et al. [2017] and Perera et al. [2022], and propose a new bidirectional program analysis framework which supports such fine-grained *in situ* provenance queries, which we call *data transparency*. Next, we use the example in Figure 2 to describe this concept. For context, the figure presents a simple piece of data visualisation, investigating the efficiency of clean energy usage, broken down geographically. The data is split across two tables dedicated to *renewable* and *non-renewable* energy sources; the data from the tables has been aggregated into a view, namely the scatter plot on the right hand side of the figure, where the x-axis

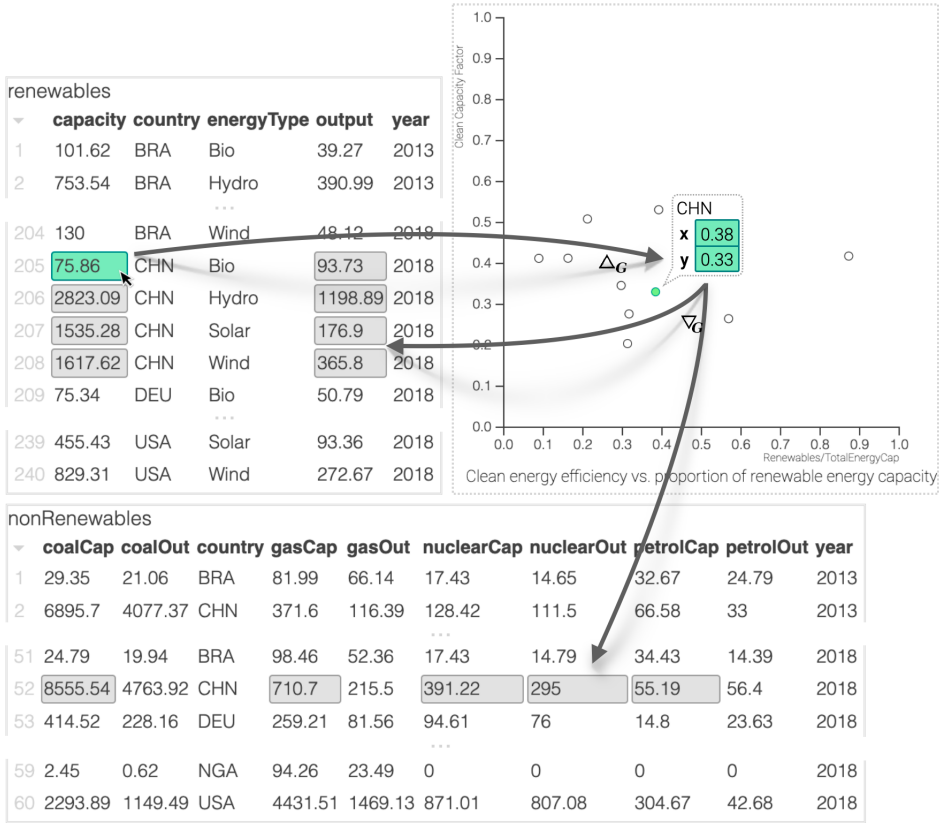


Fig. 2. Data-transparency: “What inputs does an output demand?” (bottom two arrows, provided by the ∇_G operator), “What outputs is an input demanded by?” (top arrow, Δ_G operator), and linking inputs that share a common output dependency.

represents the total renewables capacity over total energy capacity, and the y-axis denotes the clean capacity factor computed as the total clean energy output over the total clean energy capacity.

There are four scenarios below we focus on to motivate the idea of data transparency as an end-user feature. When we say “user”, this may refer to either a reader or a developer. We also often refer to the input data as simply “inputs”, and the corresponding views as “outputs”.

Fine-grained linking of the view back to the data. In Figure 2, the dots in the scatter plot do not provide any details about the data they were computed from, which may be of interest to a user. For instance, when selecting the dot highlighted in green, a user may want to identify all the values from the two tables that were used to compute its x and y coordinates (this is denoted by all the highlighted data in the two tables). Furthermore, they may be interested in more fine-grained information, e.g. they may only want to know about the clean energy, reflected in the plot by the y-axis. This is particularly opaque as the plot doesn’t make it possible to understand what “clean” energy even refers to. By selecting the y-coordinate (visible when clicking the dot), the user will be presented with a set of highlighted cells in the input tables whose contents were used to produce the selected output (all the highlighted cells in renewables, and the cells corresponding to the nuclearCap and nuclearOut columns on row 52 in nonRenewables). Notably, this clarifies that nuclear energy is considered clean energy.

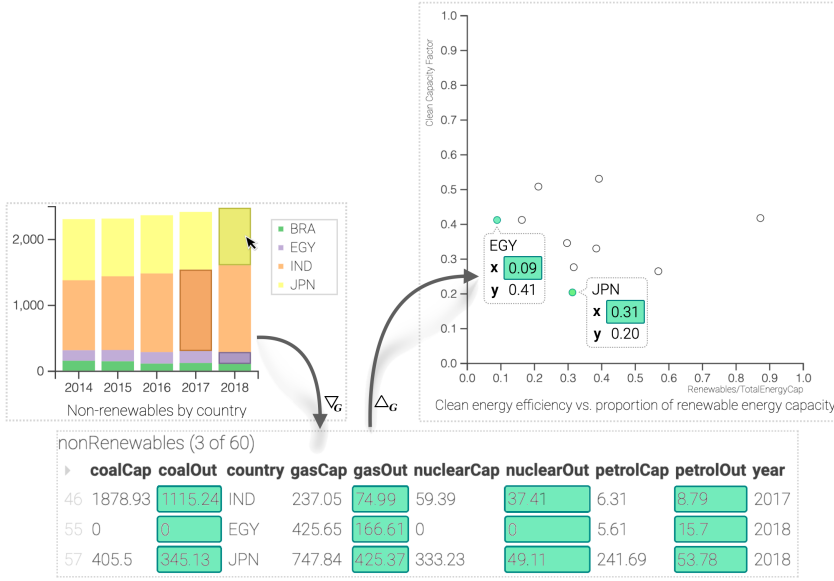


Fig. 3. Data-transparency: “What inputs does an output demand?” (left arrow, ∇_G operator), “What outputs is an input demanded by?” (right arrow, Δ_G operator), and linking outputs that share a common input dependency.

As we will see later in the paper, our system uses an operator ∇_G , which identifies the complete set of input data that was used to compute the selected output. We read this operator as computing “what input data does an output *demand*?”. The subscript G is short for “graph”, denoting that the computation is based on the reachability relation of a dependence graph G .

Fine-grained linking of the data to the view. A user may also be interested in following the link between the input data and the output view. For instance, the user may select China’s bioenergy capacity (highlighted in green in Figure 2, row 205 in the data on the figure’s left). Then, our system, through an operator we write as Δ_G , traces the data contained in this cell to the parts of the plot it was used to compute (the green dot in the plot). The bioenergy capacity contributes to both components of the dot, namely the total renewables capacity over total energy capacity and the clean capacity factor. We read Δ_G as computing “what outputs is some input data *demand*ed by?”.

Linked inputs. The previous use cases only went in one direction, either from outputs back to inputs, or from inputs to outputs. However, there may be situations where we want to compose the two directions. For illustration, let’s assume like before that a user starts by selecting China’s bioenergy capacity. As previously explained, our system will apply Δ_G to trace the data contained in this cell to the point in the scatter plot it was used to compute (the green dot in the plot). Additionally, our system will apply ∇_G to determine the rest of the data in the input tables that was used to compute the point in the plot (all data highlighted in grey in the tables). We refer to the highlighted data in the two input tables as “linked inputs”, i.e., inputs that contribute to the same features of an output. This enables users to formulate queries like: “which outputs use this data element, and which other data elements are used along with it?”.

Linked outputs. Dually, we can start from an output, and want to determine all the “linked outputs”, i.e., outputs that compete for the same input data. One such scenario is illustrated in

Figure 3 where we have two output views: the same scatter plot as in Figure 2, and an additional bar chart denoting the non-renewables by country. Let's assume that a user selects the segments corresponding to India in 2017, and Egypt and Japan in 2018 in the bar chart. This will highlight the corresponding data in the nonRenewables table that was used to produce those outputs. Additionally, it will also highlight any outputs in the scatter plot that demand any of those data. The selected bars in the bar chart and the selected points (specifically, their x-coordinates as those are the ones using the considered input data) in the scatter plot are the linked outputs. In the scatter plot, we note that only two dots are selected, those for Egypt and Japan, reflecting the fact that the scatter plot is only a view of data from 2018.

From the analysis point of view, this requires applying ∇_G to find the value used to produce the selected segment of the bar, with each set of coloured segments associated with a given nation – the selected segments demand all of the highlighted green data in the table. Then, starting from the selected data, we apply Δ_G to pick out the x components of some of the circles in the scatter plot (green circles).

1.2 Contributions

We introduce a new bidirectional program analysis framework which supports interactive exploration of fine-grained I/O relationships, as described in the previous section. We make use of dynamic dependence graphs [Ferrante et al. 1987; Agrawal and Horgan 1990]; this enables our approach to be language-independent, separating the definition of queries over the dependence graph for a particular program, from the problem of deriving the dependence graph for that program. Our specific contributions are as follows:

- A new formal framework for bidirectional provenance queries over dependence graphs, formalising the ∇_G and Δ_G operators introduced informally in § 1.1, which we show to be *conjugate* in the sense of Jonsson and Tarski [1951]. We relate our framework to bidirectional analyses based on Galois connections (e.g. Ricciotti et al. [2017]; Perera et al. [2022]).
- A novel *linked inputs* analysis, a relation of mutual relevance or “cognacy” which arises between inputs when they contribute to common features of the output. The design of our framework makes it straightforward to implement this analysis by composing ∇_G and Δ_G . We also show how a dual *linked outputs* analysis, available in prior work [Perera et al. 2022], is easily obtained by transposing the two operators.
- Improved performance with respect to *trace-based* approaches [Perera et al. 2022], one of the predominant techniques for achieving bidirectional program analysis. Using usability metrics from Nielsen [1994], we find that 81% of the queries we tested over larger examples execute at a speed that appears instantaneous to a user, compared to 25% using an implementation based on traces. Fast response time is particularly important for the interactive data visualisations we target.

These contributions are not available in previous works. Hand-crafted efforts like Bremer and Ranzijn [2015]’s are not only labour-intensive, but involve manually embedding metadata about the relationship between visual outputs and inputs into the same program. More automated, trace-based, solutions like Ricciotti et al. [2017] and Perera et al. [2022], which shift that responsibility into the programming language, use a bidirectional interpreter that is costly both in terms of implementation effort and performance. For example, these languages are difficult to update, with each change requiring both directions of the interpreter to be modified in a way that maintains bidirectionality; and each new analysis involves re-running the entire program backwards, regardless of whether all of it is relevant.

Roadmap. The paper is organised as follows:

- § 2 presents an informal overview of our approach and language.
- § 3 defines a new program analysis framework over dynamic dependence graphs, introducing the cognacy operator $\nabla\Delta_G$ (“linked inputs”) and unpacking the intuitive relationship between its two components ∇_G (“demands”) and Δ_G (“demanded by”) in terms of Jonsson and Tarski’s notion of conjugate operators over Boolean algebras. We also introduce the dual cognacy operator $\Delta\nabla_G$ (“linked outputs”) and explain the relationship to Galois connections: f and g being conjugate is equivalent to f and the De Morgan dual of g forming a Galois connection. We give procedures for computing ∇_G and Δ_G .
- § 4 defines a core functional language with an operational semantics that pairs every result with a dynamic dependence graph suitable for computing ∇_G and Δ_G . We show how to represent (parts of) values as vertices in the graph. We also show how our operators can follow information back to environments, with projections.
- § 5 evaluates our implementation based on graphs and conjugates against one based on traces and Galois connections. We contrast the implementation burden of the two approaches, and the performance of building trace vs. building dependence graphs and computing ∇_G and Δ_G over each.
- § 6 reviews related work in more detail, including program slicing and dynamic dependence graphs.
- § 7 wraps up with a discussion of some limitations. In particular, the sort of transparency infrastructure proposed in this paper is purely extensional and falls short of providing full explanations of how output parts are related to input parts. We discuss extending this to more “intensional” forms transparency and propose some other ways in which the present system could be improved.

We provide examples of practical programs written in our framework in the supplementary material, including those used to generate Figure 2 and Figure 3. Our implementation is open source and available (given later in a deanonymised version). To support our claims about fast response times, we also host an anonymised interactive web demo at <http://opencomputation.org/> (the user interface doesn’t yet support selecting individual coordinates for a dot in the scatter plot, although this is available in our core implementation).

2 INFORMAL OVERVIEW OF THE SOLUTION

2.1 Conjugate Operators over Dependence Graphs

Here, we provide an informal view of our framework for bidirectional provenance queries over dependence graphs, which will be followed by a formal account in § 3. As previously mentioned, our insight in this paper is that dependence graphs provide a language-agnostic foundation for data transparency that both improves performance and reduces the implementation burden.

The operators ∇_G and Δ_G , introduced in § 1 to perform queries over a dependence graph, form what’s called a “conjugate pair”. Conjugate pairs – which map between Boolean algebras, e.g., with carrier sets X and Y – have the property of maintaining “overlap”. If the result of applying one member of the pair to some subset of inputs $X' \subseteq X$ does not overlap with $Y' \subseteq Y$, then applying the other member of the pair to Y results in a set which does not overlap with X' . This property intuitively holds of our dependency queries: if no member of a given input selection is demanded by a given subset of the outputs, then that subset of outputs will not demand any member of the given inputs.

The canonical example of conjugate operations are the image and preimage functions $R[\cdot]$ and $R^{-1}[\cdot]$ for a relation $R \subseteq X \times Y$. It is not hard to see that if $R^{-1}[Y']$ overlaps with X' where $X' \subseteq X$

then necessarily $R[X']$ overlaps with Y' where $Y' \subseteq Y$, and vice versa. Conjugate pairs are also closely related to Galois connections; if f and g are conjugate, then f and the De Morgan dual of g form a Galois connection (which will be explained in more detail in § 3).

If we now take R to be a reachability relation in a graph G , then technically, the graph setting only requires a single operator ∇_G – the “demands” relation (of outputs on inputs) – to support all the required IO queries. Given a procedure to compute ∇_G , we can automatically derive a procedure to compute its conjugate Δ_G – the “demanded-by” relation (of inputs for outputs) – by simply using the opposite graph G^{-1} ’s reachability relation, $\nabla_{G^{-1}}$. By then using the De Morgan Dual, one can easily derive a new operator $\blacktriangle_G$ which can be used to investigate all questions of “sufficiency” of inputs for outputs; that is, given some inputs, what outputs are they sufficient for computing. (It is well-known that extensionally any one of these is enough to determine each other, but in the graph setting we also benefit from efficient procedures for all of them.) In practice however, using the De Morgan dual construction – i.e. complementing an initial input selection, running Δ_G , and complementing the resulting output selection – is in general inefficient, since complement operations are expensive. Instead, we will provide a more efficient implementation (of Δ_G and $\blacktriangle_G$), which exhibits significantly better performance compared to the derived algorithm.

In the graph setting, the “linked inputs” and “linked outputs” relations can be computed by composing reachability in G with reachability in G^{-1} , which enables an efficient implementation. For example, linked inputs can be written as $\nabla\Delta_G := \nabla_G \circ \Delta_G$ (i.e. $\nabla_G \circ \nabla_{G^{-1}}$), and linked outputs can be written as $\Delta\nabla_G := \Delta_G \circ \nabla_G$. In the graph, a pair of linked output values share some common ancestor whereas a pair of linked input nodes in the graph share some common descendant.

As a mnemonic device to remember the meaning of the symbols here, consider traces through a dependence graph G from inputs coming in *at the top* flowing down to outputs *at the bottom*. Calculating the *demands* ∇_G relates outputs at the bottom (the point of the triangle) to some inputs at the top (the flat triangle base)—the demands of an output. Dually, calculating *demanded by* Δ_G relates inputs at the top (the point of the triangle) to some outputs at the bottom (the triangle base).

2.2 [OurLanguage]: A Data-Transparent Programming Language

We implement the abstract framework outlined above in a programming language called [OurLanguage], a pure functional language with a runtime implementing conjugate dependency operators. From the programmer’s perspective the language is unremarkable. The distinguishing feature of the language is a special semantics that interprets a program not as a plain output but as an output with a built-in query engine supporting the transparency features shown above. The author of a visualisation just expresses their chart as a pure function of the inputs, via a set of data types for common visualisations; a d3.js front end automatically enriches the rendered outputs with support for visual selection and various selection-based queries.

We set out the formal graph framework in § 3, and then show how our core language [OurLanguage] implements that abstract framework in § 4.

3 CONJUGATE OPERATORS FOR DYNAMIC DEPENDENCE GRAPHS

In this section we introduce Boolean algebras and other preliminaries (§ 3.1); dynamic dependence graphs (§ 3.2), and the image and preimage functions for a relation and their De Morgan duals (§ 3.3). We then extend these operators to dependence graphs (§ 3.4) via their IO relation, and give algorithms for computing them over a given graph (§ 3.6). Throughout the section, we will illustrate concepts using Figure 4 as a running example, showing a toy program and (a simplified version of) its dynamic dependence graph that represents the program’s execution.

3.1 Preliminaries

We give some notation used in the rest of the paper. X, Y range over finite sets.

Finite sequences. Write $\vec{x}$ denote a finite sequence of elements $x_1, \dots, x_n$, with ϵ as the empty sequence. Concatenation of two sequences $\vec{x}$ and $\vec{x}'$ is written $\vec{x} ++ \vec{x}'$. We also write $x \cdot \vec{x}$ for cons (prepend) and $\vec{x} \cdot x$ for snoc (append).

Finite maps. Write $\overrightarrow{\{k : x\}}$ to denote a set of pairs $k_1 : x_1, \dots, k_n : x_n$ where keys k_i are pairwise unique.

Disjoint union. Write $X \uplus Y$ to mean $X \cup Y$ where X and Y are disjoint, and also write $x \cdot X$ or $X \cdot x$ to mean $X \uplus \{x\}$. For finite maps ϕ, ψ write $\phi \uplus \psi$ to mean $\phi \cup \psi$ where $\text{dom}(\phi)$ and $\text{dom}(\psi)$ are disjoint.

Relative complement. Write $X \setminus Y$ for the relative complement of Y in X , i.e. the set of elements of X that are not in Y .

Relational converse. Write $\cdot^{-1}$ for the involutive operation that takes a relation $R \subseteq X \times Y$ to its converse $R^{-1} \subseteq Y \times X$ where $(y, x) \in R^{-1} \iff (x, y) \in R$.

3.1.1 Boolean algebras. A *Boolean algebra* (or Boolean lattice) is a 6-tuple $A = (A, \wedge, \vee, \perp, \top, \neg)$ with carrier A , distinguished elements bottom ($\perp$) and top ($\top$), commutative operations meet ($\wedge$) and join ($\vee$) with $\top$ and $\perp$ as respective units, and a unary operation negate ($\neg$) satisfying $x \vee \neg x = \top$ and $x \wedge \neg x = \perp$. The operations $\wedge$ and $\vee$ distribute over each other and induce a partial order $\leq$ over X .

Often we will be considering a specific finite Boolean algebra, namely the powerset $\mathbb{P}(X)$ whose carrier is the set of subsets $X' \subseteq X$ ordered by inclusion $\subseteq$. Bottom is the empty set $\emptyset$, top is the whole set X , meet and join are given by $\cap$ and $\cup$ and negation by relative complement, so that $\neg X' = X \setminus X'$. For a program, we will usually consider the powerset of nodes associated with input values and the powerset of nodes associated with outputs.

Example. In Figure 4, the powerset of input nodes and output nodes are $\mathbb{P}(\{\alpha_1, \alpha_2, \alpha_3\})$ and $\mathbb{P}(\{\alpha_4, \alpha_5, \alpha_6\})$ respectively.

3.2 Dynamic Dependence Graphs

We explain how we construct dependence graphs later in § 4, and here just introduce the basic notion. A *dynamic dependence graph* [Agrawal and Horgan 1990] is a directed acyclic graph $G = (V, E)$ with a set V of *vertices* and a set $E \subseteq V \times V$ of edges. In the dependence graph for a particular program, vertices $\alpha, \beta \in V$ represent *parts* of values (either supplied to the program as inputs or produced during evaluation) and edges $(\alpha, \beta) \in E$ indicate that, in the evaluation of the program, the partial value associated to β *depends on* the partial value associated to α . (The notion of a “part” of a value will be made precise in § 4.) This diverges somewhat from traditional approaches to dynamic dependence graphs in only considering one type of edge (data dependency) rather than separate data and control dependencies; moreover our edges point in the direction of dependent vertices, whereas in the literature the other direction is somewhat more common.

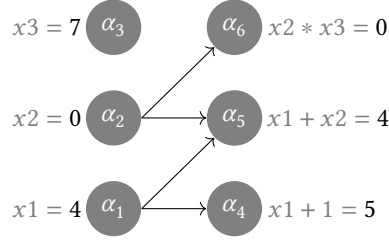
Example. In Figure 4b, the nodes $\{\alpha_1, \alpha_2, \alpha_3\}$ are associated to the values of the input variables x_1, x_2, x_3 in Figure 4a, and the nodes $\{\alpha_4, \alpha_5, \alpha_6\}$ are associated with the values of the arithmetic expressions in the program’s result.


```

1  let x1 = 4;
2    x2 = 0;
3    x3 = 7
4  in (x1 + 1, (x1 + x2, x2 * x3))

```

(a) Program



(b) Dynamic dependence graph

Fig. 4. Running example: a source program and its corresponding dynamic dependence graph

Definition 3.1 (In-edges and out-edges). For a graph $G = (V, E)$ and vertex $\alpha \in V$, write $\text{in}_G(\alpha)$ for the in-edges of α in G and $\text{out}_G(\alpha)$ for its out-edges.

Definition 3.2 (Sources and sinks). For a graph $G = (V, E)$, write $S(G)$ for the *sources* of G (i.e. those vertices with no in-edges) and $T(G)$ for the *sinks* of G (those with no out-edges).

Definition 3.3 (Reachability relation). Define the *reachability relation* for a graph $G = (V, E)$ to be the reflexive transitive closure of E .

Definition 3.4 (Opposite graph). For a graph G define the *opposite graph* $G^{-1} := (V, E^{-1})$ where $\cdot^{-1}$ denotes relational converse.

(Taking the opposite graph has the effect of swapping sources and sinks.)

3.2.1 IO Relation for a Graph. Reachability in G induces a relation just between sources and sinks, which we call the *IO relation* of G .

Definition 3.5 (IO relation). For any dependence graph G with reachability relation R , define the *IO relation* of G to be $R \cap (S(G) \times T(G))$.

The IO relation specifies how specific inputs (sources) are demanded by specific outputs (sinks); thus we will interpret $(x, y) \in R$ as “ x is *demanded by* y ”.

Example. In Figure 4b, the program is simple enough that the reachability relation and the IO relation are identical.

Clearly the IO relation of G is the converse of the IO relation of G^{-1} . Since we are in fact interested in the relationship between input and output *selections* (namely sets of inputs and outputs), we now consider how such a relation $R \subseteq X \times Y$ lifts to $\mathbb{P}(X)$ and $\mathbb{P}(Y)$ via the *image* and *preimage* functions for R , which we will denote using Δ_R and ∇_R .

3.3 Conjugate Image and Preimage Functions

Definition 3.6 (Image and Preimage Functions for a Relation). For a relation $R \subseteq X \times Y$, define $\Delta_R : \mathbb{P}(X) \rightarrow \mathbb{P}(Y)$ and $\nabla_R : \mathbb{P}(Y) \rightarrow \mathbb{P}(X)$ as:

- (1) $\Delta_R(X') := \{y \in Y \mid \exists x \in X'. (x, y) \in R\}$ (image)
- (2) $\nabla_R(Y') := \{x \in X \mid \exists y \in Y'. (x, y) \in R\}$ (preimage)

It is well known that the image function for R is the preimage function for its converse; this is important enough to state as a lemma:

LEMMA 3.7 (IMAGE AS PREIMAGE OF CONVERSE RELATION). $\Delta_R = \nabla_{R^{-1}}$

When thinking of R as an IO relation, the image $\Delta_R(X')$ consists of all outputs that any element of X' is demanded by, and the preimage $\nabla_R(Y')$ consists of all inputs demanded by any element of Y' . If R happens to be a function, the image and preimage functions form a *Galois connection* (Definition 3.14 below); for a proper relation they form a *conjugate pair*, in the sense of Jonsson and Tarski [1951]:

Definition 3.8 (Conjugate Functions). For Boolean algebras A, B , functions $f : A \rightarrow B$ and $g : B \rightarrow A$ form a *conjugate pair* iff:

$$f(x) \wedge y = \perp \iff x \wedge g(y) = \perp$$

(Jonsson and Tarski consider only endofunctions; here we extend the idea of conjugacy to maps between Boolean algebras.)

LEMMA 3.9 (PREIMAGE AND IMAGE OF A RELATION ARE CONJUGATE). *For any relation $R \subseteq X \times Y$ the functions Δ_R and ∇_R are conjugate.*

This should be intuitive enough: for any subsets $X' \subseteq X$ and $Y' \subseteq Y$, if the elements “on the right” to which X' is related are disjoint from Y' , then there are no edges in R from X' to Y' ; and in virtue of that, the elements “on the left” to which Y' is related must also be disjoint from X' .

3.4 ∇_G and Δ_G for Dependence Graphs

We now extend the notion of ∇_R and Δ_R to a dependence graph G via its IO relation. The set of inputs that some outputs Y *demand*, ∇_G , is simply the subset of the inputs of G reachable from Y (by traversing the graph in its opposite direction). Conversely, the set of outputs *demand*ed by some inputs X , Δ_G , is simply the subset of the outputs of G reachable from X .

Definition 3.10 (∇_G and Δ_G for a dependence graph). For a graph G with IO relation R , define:

- (1) $\nabla_G := \nabla_R : \mathbb{P}(T(G)) \rightarrow \mathbb{P}(S(G))$ (demands)
- (2) $\Delta_G := \Delta_R : \mathbb{P}(S(G)) \rightarrow \mathbb{P}(T(G))$ (demanded by)

Example. In Figure 4, if we first consider *demanded by*, then $\Delta_G(\{\alpha_1\}) = \{\alpha_4, \alpha_5\}$ because the value of x_1 contributes to the values associated at those nodes. Conversely, if we analyse the *demands* of those nodes, then $\nabla_G(\{\alpha_4, \alpha_5\}) = \{\alpha_1, \alpha_2\}$ since the value of x_2 is also used in the construction of the value at node α_5 . On the other hand, taking $\nabla_G(\{\alpha_6\})$ would only return $\{\alpha_2\}$, despite α_6 representing the value of the expression $x_2 * x_3$; this is because $x_2 = 0$, so the value of x_3 has no effect on the result.

3.5 Dual Image and Preimage Functions

If R is an IO relation, then $\Delta_R(X)$ effectively asks “what is X necessary for?” This naturally leads to the question “what is X *sufficient* for?” In fact, we can use the notion of sufficiency as a test of correctness. If we begin by selecting an output of interest, we would expect the data it demands to also be sufficient to reconstruct it. This leads us to define the following pair of functions, which also form a conjugate pair.

Definition 3.11 (Dual Image and Preimage Functions for a Relation). For any relation $R \subseteq X \times Y$, define $\blacktriangle_R : \mathbb{P}(X) \rightarrow \mathbb{P}(Y)$ and $\blacktriangledown_R : \mathbb{P}(Y) \rightarrow \mathbb{P}(X)$ as the composition:

$$(1) \blacktriangle_R(X') := \{y \in Y \mid \nexists x \in X \setminus X'. (x, y) \in R\} \quad (\text{dual image})$$

$$(2) \blacktriangledown_R(Y') := \{x \in X \mid \nexists y \in Y \setminus Y'. (x, y) \in R\} \quad (\text{dual preimage})$$

This definition is similar to Δ_R except that $\blacktriangle_R$ needs every ancestor to be in the input set, rather than any ancestor to be in the input set. We include $\blacktriangledown_R$ here only for completeness; although it is derived/implemented as easily as $\blacktriangle_R$, we so far find its application unimportant for the purpose of IO-related queries, so ignore it hereon.

Example. If we apply $\blacktriangle_R$ to dependence graphs, i.e. $\blacktriangle_G$, then in Figure 4 we have that $\blacktriangle_G(\{\alpha_1\}) = \{\alpha_4\}$, contrasting with $\Delta_G(\{\alpha_1\}) = \{\alpha_4, \alpha_5\}$ before. Although the value at α_1 is necessary to compute the value at α_5 , we have not selected the node α_2 as an input to $\blacktriangle_G$, and so have insufficient information to compute the value at α_5 .

Notably, Δ_R can be transformed into $\blacktriangle_R$ by using the De Morgan Dual:

Definition 3.12 (De Morgan Dual). Suppose a function $f : A \rightarrow B$ between Boolean algebras with $\neg_A$ and $\neg_B$ the negation operators of A and B . Define the *De Morgan Dual* $f^\circ : A \rightarrow B$ of f as:

$$f^\circ := \neg_B \circ f \circ \neg_A$$

LEMMA 3.13 (DUALITY OF IMAGE AND PREIMAGE FUNCTIONS).

$$(1) \Delta_R^\circ = \blacktriangle_R$$

$$(2) \nabla_R^\circ = \blacktriangledown_R$$

PROOF. See the supplementary material □

Intuitively, since $\neg$ for powersets is just complementation, c , we can read $\Delta_R(X^c)$ as “what outputs use unselected input data?”. Then, taking the complement of this to get $\Delta_R(X^c)^c$ is effectively asking the question “what outputs do not use any of the information I have not selected?”, which is exactly the question of sufficiency.

Conjugate functions are related to another class of functions between Boolean algebras, called Galois connections. Specifically, a well known result in the literature [Menni and Smith \[2014\]](#) is that every conjugate pair of functions induces a Galois connection:

Definition 3.14 (Galois connection). Suppose A, B are partial orders. Then monotone functions $f : A \rightarrow B$ and $g : B \rightarrow A$ form a *Galois connection* iff

$$f(x) \leq y \iff x \leq g(y)$$

PROPOSITION 3.15 (CORRESPONDENCE BETWEEN CONJUGATES AND GALOIS CONNECTIONS). *Suppose functions $f : A \rightarrow B$ and $g : B \rightarrow A$ between Boolean algebras. The following statements are equivalent:*

(1) *f and g form a conjugate pair*

(2) *f and g° form a Galois connection*

3.6 Computing ∇_G , Δ_G , and $\blacktriangle_G$ for Dependence Graphs

We now show how to compute ∇_G , Δ_G , and $\blacktriangle_G$ for a dependence graph G . One advantage of graphs, in comparison to the trace-based approach mentioned in § 1, is that a single algorithm can be readily adapted to compute all 3 operators; for example, given a procedure for Δ_G , we can compute its conjugate ∇_G as $\Delta_{G^{-1}}$ (by Lemma 3.7) and its dual $\blacktriangle_G$ by pre- and post-composing with negation (Definition 3.12).

For efficiency, however, we give a direct procedure both for Δ_G (§ 3.6.1) and $\blacktriangle_G$ (§ 3.6.2) in Figure 5, where ∇_G is easily computed by performing Δ_G on the opposite graph; this is in some

situations more efficient than relying on the De Morgan dual, which can cause more graph to be traversed than would be otherwise. Both procedures factor through an auxiliary operation that computes a “slice” of the original graph, i.e. a contiguous subgraph. While it is technically possible to compute the desired image/preimage of the IO relation without creating this intermediate graph, we anticipate use cases which will make use of the graph slice; these are discussed in more detail in § 7. Our implementation makes it easy to flip between G and G^{-1} so we freely make use of both in the algorithms to access out-neighbours and in-neighbours.

3.6.1 Direct algorithm for Δ_G (demanded by).

Definition 3.16 (demBy_G). Figure 5a defines the family of relations demBy_G for dependence graph G .

For a dependence graph G and inputs $X \subseteq S(G)$, the judgement $X \text{ demBy}_G Y$ says that X is demanded by $Y \subseteq T(G)$. It is possible to read the algorithm directly from the inductive definition, i.e. treating it as a G -indexed function that takes X as input.

The algorithm defers to an auxiliary operation “ $H, G \text{ demBy} V$ ”, which maintains a (partial) graph slice $H \subseteq G$, where H initially contains only the original input vertices X and no edges. It then proceeds as follows. If there is a sink of H that still has outgoing edges in G , then the targets of those edges are reachable from the vertices of H , and so we must add them to H and recurse (extend). If $T(H) \subseteq T(G)$, then there are no unexplored nodes in G that are reachable from H , and so we terminate with the current state of H (done). We note that it is enough to consider the sinks of H , since we move every outgoing edge of a vertex from G to H all at once. When demBy_G is done, demBy_G returns the sinks $T(H)$ from the final graph slice H , representing the outputs that the original inputs are demanded by.

PROPOSITION 3.17 (demBy_G COMPUTES Δ_G). For any dependence graph G , any $X \subseteq S(G)$ and any $Y \subseteq T(G)$ we have

$$X \text{ demBy}_G Y \iff \Delta_G(X) = Y$$

PROOF. See the supplementary material □

3.6.2 Direct algorithm for $\blacktriangle_G$ (suffices).

Definition 3.18 (suff_G). Figure 5b defines the family of relations suff_G for dependence graph G .

For a dependence graph G , the algorithm suff_G also delegates to an auxiliary operation suffE which builds a graph slice H , similar to demands. The operation “ $H, P, G \text{ suffE}$ ” takes a graph slice H under construction, a “pending” subgraph P of nodes for which we have discovered partial information, and the remaining unexplored graph G . Whenever we have a vertex in H which still has outgoing edges in G , we add those edges and their endpoints to the pending graph P (pending). If a vertex α has no more incoming edges in G , we can then move α and its incoming edges from P to H (extend), as this only happens when we have already moved every ancestor of α into H . If neither scenario is the case, then we terminate with the (potentially incomplete) graph H (done). Once suffE has terminated, suff_G takes H , and uses intersection to extract the vertices in H that are also sinks in G . If this intersection is empty, it simply means that the input data X was insufficient to compute any of the original program’s outputs.

We provide an example of a run of the algorithm in Figure 6. Here, the green vertices and thick edges are in H , the orange vertices and dashed arrows are in P , and the blue vertices and normal arrows are in G . In the figure, steps 2, 3, 5 and 6 correspond to applications of the rule pending, and steps 4 and 7 correspond to applications of the rule extend. After step 7, the run is complete.

$X \text{ demBy}_G Y$

$$\frac{(X, \emptyset), G \text{ demByV } H}{X \text{ demBy}_G T(H)} X \subseteq S(G)$$

$H, G \text{ demByV } H'$

done

$$\frac{}{H, G \text{ demByV } H} T(H) \subseteq T(G)$$

extend

$$\frac{H \cup E, G \setminus_E E \text{ demByV } H'}{H, G \text{ demByV } H'} \alpha \in T(H) \wedge E = \text{outE}_G(\alpha) \neq \emptyset$$

(a) demBy_G algorithm

$X \text{ suff}_G U$

$$\frac{(X, \emptyset), (\emptyset, \emptyset), G \text{ suffE } H}{X \text{ suff}_G V(H) \cap T(G)} X \subseteq S(G)$$

$H, P, G \text{ suffE } H'$

done

$$\frac{}{H, P, G \text{ suffE } H} S(G) \cap V(P) \subseteq S(P) \wedge V(H) \subseteq T(G)$$

pending

$$\frac{H, P \cup E, G \setminus_E E \text{ suffE } H'}{H, P, G \text{ suffE } H'} \alpha \in V(H) \wedge E = \text{outE}_G(\alpha) \neq \emptyset$$

extend

$$\frac{H \cup E, P \setminus_E E, G \text{ suffE } H'}{H, P, G \text{ suffE } H'} \alpha \in S(G) \wedge E = \text{inE}_P(\alpha) \neq \emptyset$$

(b) suff_G algorithm

Fig. 5. Dual analyses over a dependence graph G

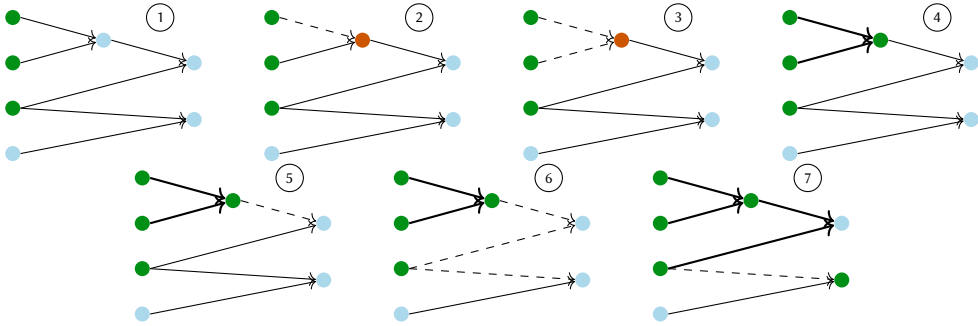


Fig. 6. Run of suff with G and H superimposed on G_0

PROPOSITION 3.19 (suff_G COMPUTES $\blacktriangle_G$). *For any dependence graph G , any $X \subseteq S(G)$ and any $Y \subseteq T(G)$ we have:*

$$X \text{ suff}_G Y \iff \blacktriangle_g(X) = Y$$

PROOF. See the supplementary material □

3.6.3 Derived algorithms. Propositions 3.17 and 3.19 justify treating demBy_G and suff_G as functions. Using Lemma 3.13 we can define a direct algorithm for ∇_G from demBy_G by simply flipping the graph. (The same could be done to acquire an algorithm for $\blacktriangledown_G$ from suff_G .) Using Lemma 3.13 and Definition 3.12 we can also define alternative algorithms for all 3 operators using the De Morgan dual construction. All algorithms are summarised in Figure 7.

Abstract operator	Direct algorithm	Alternative algorithm
Δ_G	demBy_G	suff_G°
∇_G	$\text{demBy}_{G^{-1}}$	$\text{suff}_{G^{-1}}^\circ$
$\blacktriangle_G$	suff_G	demBy_G°

Fig. 7. Direct and alternative (De Morgan Dual) algorithms for all 3 operators

In § 5 we contrast the performance of the direct and De Morgan dual implementations of Δ_G , which is an essential component of the linked inputs and linked outputs operators introduced in § 2.

3.7 $\nabla\Delta_G$ and $\Delta\nabla_G$ for Dependence Graphs

Now we can provide a formal account of the notions of linked inputs and linked outputs.

Definition 3.20 (Linked inputs and outputs). For a dependence graph G define:

- (1) $\nabla\Delta_G := \nabla_G \circ \Delta_G$ (linked inputs)
- (2) $\Delta\nabla_G := \Delta_G \circ \nabla_G$ (linked outputs)

Intuitively, linked outputs and linked inputs are relations of *cognacy* (common ancestry) in G and G^{-1} respectively. For a set of inputs $X \subseteq S(G)$, linked inputs asks “what other inputs are demanded by the outputs which demand X ?”; it first finds all outputs $\Delta_G(X)$ that our inputs are demanded by, and then computes the inputs that those outputs demand, i.e. $\nabla_G(\Delta_G(X))$. Conversely for a set of outputs $Y \subseteq T(G)$, linked outputs asks “what other outputs demand the inputs that Y demands”; it first finds all inputs $\nabla_G(Y)$ that our outputs demand, and then computes the outputs that those inputs $\nabla_G(Y)$ demand, i.e. $\Delta_G(\nabla_G(Y))$.

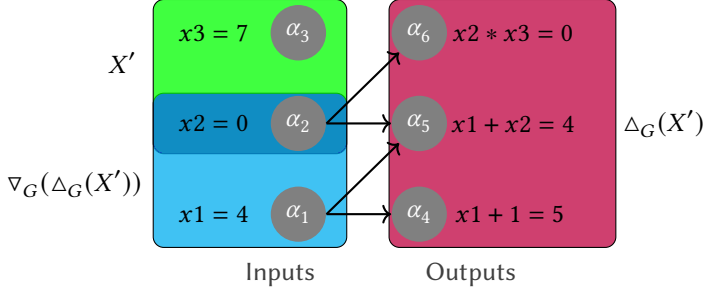
In general, neither $\nabla\Delta_G$ nor $\Delta\nabla_G$ are necessarily monotonic. We illustrate this for linked inputs $\nabla\Delta_G$ in Figure 8, which uses the graph from the example in Figure 4b. In the figure, the top-left vertex is part of the initial input selection X' , but is not preserved by the $\nabla_G \circ \Delta_G$ round-trip because it is not demanded by any output. Thus we have neither $X' \subseteq \nabla\Delta_G(X')$, nor $X' \supseteq \nabla\Delta_G(X')$. It is not hard to see how the same concept arises for linked outputs, by using $\Delta_G \circ \nabla_G$ on the graph of a program that outputs a constant.

If we constrain things so that there are no inputs which are unused, then ∇_G will preserve $\perp$, i.e. $\nabla_G(\emptyset) = \emptyset$, and $\Delta\nabla_G$ will preserve the original output selection. Similarly, if we constrain things so that all outputs demand some input, then Δ_G will preserve $\perp$, and $\nabla\Delta_G$ will preserve the original input selection:

LEMMA 3.21.

- (1) If $\Delta_G(\emptyset) = \emptyset$, then for all X we have $\nabla\Delta_G(X) \geq X$.
- (2) If $\nabla_G(\emptyset) = \emptyset$, then for all Y we have $\Delta\nabla_G(Y) \geq Y$.

As an easy corollary, if ∇_G and Δ_G both preserve $\emptyset$ then in fact they form an antitone Galois connection. In practical terms, it is possible to use this to produce a filtered view, only showing inputs that are actually used in a program, which has the potential to make exploring a visualisation a smoother experience for an end user.

Fig. 8. Linked inputs ∇_{Δ_G} is not necessarily monotonic

Expression		Eliminator	
$e ::= x$	variable	$\sigma ::= x \mapsto \kappa$	variable
n	integer	$\{\vec{x}\} \mapsto \kappa$	record
$\text{let } x = e \text{ in } e'$	let	$\{c \mapsto \vec{k}\}$	constructor
$\{\vec{x} : \vec{e}\}$	record	Value	
$e.x$	record projection	$v ::= v_\alpha$	
$c(\vec{e})$	constructor	Raw value	
$e e'$	application	$v ::= n$	integer
$f(\vec{e})$	foreign application	$\{\vec{x} : \vec{e}\}$	record
$\lambda \sigma$	function	$c(\vec{e})$	constructor
$\text{let } \vec{x} : \vec{\sigma} \text{ in } e$	recursive let	$\text{cl}(\gamma, \rho, \sigma)$	closure
Continuation		Environment	
$\kappa ::= e$	expression	$\gamma ::= \{\vec{x} : \vec{e}\}$	
σ	eliminator	Recursive definitions	
		$\rho ::= \{\vec{x} : \vec{\sigma}\}$	

Fig. 9. Syntax of core language, including annotated values

4 A CORE LANGUAGE WITH GRAPH SYNTAX AND SEMANTICS

In § 3, we defined the abstract setting of dependence graphs and conjugate operators *demands* (∇) and *demanded by* (Δ) over such graphs. We now implement that graph abstraction for [OurLanguage], an untyped, pure functional programming language. This core calculus we go on to describe is what underlies the surface-language programs used to generate and analyse the scatter plots from earlier; we provide these programs and other practical examples in the supplementary material. First, we introduce the syntax of our core language constructs (§ 4.1). Then we define a big-step operational semantics that simultaneously (i) evaluates a term to a value and (ii) constructs an accompanying dependence graph for the evaluation of that term (§ 4.2); we also address how we deal with pattern-matching and mutual recursion.

4.1 Syntax

4.1.1 Terms. Figure 9 defines the expressions e of the language, which include variables x, y, f ; integer constants n, m ; let-bindings $\text{let } x = e \text{ in } e'$; record construction $\{\vec{x} : \vec{e}\}$; and record

projection $e.x$. We also have constructor expressions $c(\vec{e})$, where c ranges over data constructors and is fully saturated by arguments $\vec{e}$; we write $\Sigma(c) = n$ to denote that c takes $n \in \mathbb{N}$ arguments. Application comes in two forms: standard application $e e'$, and *foreign application* $f(\vec{e})$ for calling foreign functions which are explained in § 4.1.2. The last two terms define anonymous functions $\lambda\sigma$ and (mutually) recursive let-bindings $\text{let } \vec{x} : \vec{\sigma} \text{ in } e$, where σ is a pattern-matching construct called an *eliminator* detailed further in § 4.1.3.

4.1.2 Foreign functions. The language is parameterised by a finite map $\Phi(f) = n$ from variables f to the arities $n \in \mathbb{N}$ of the foreign function they represent (analogous to $\Sigma(c) = n$ for constructors). For the graph semantics that will follow, every foreign function f is required to provide an interpretation $\hat{f}$ that computes both f 's result and a suitable dependence graph.

Foreign functions in the core language are not first-class and calls $f(\vec{e})$ are saturated, i.e. $|\vec{e}| = \Phi(f)$. In our implementation, which also provides a surface language which desugars into the core, first-class foreign functions are supported via a top-level environment which maps every f to the closure $\text{cl}(\emptyset, \epsilon, x_1 \mapsto \dots \mapsto x_{\Phi(f)} \mapsto f(\vec{x}))$ where $|\vec{x}| = \Phi(f)$.

4.1.3 Continuations and eliminators. A *continuation* κ describes how an execution proceeds after a value is pattern-matched, and is either a term e or an *eliminator* σ . Eliminators are trie-like objects [Hinze 2000] that specify how this pattern-matching is done and the subsequent continuation κ to be executed under additional variable bindings. These are what the piecewise function definitions of our surface-language, e.g. `filter` in Figure 13a, desugar into; Peyton Jones et al. [2022] present a similar idea. (The use of eliminators in our language makes extracting dependency information easier in our implementation, and goes well with records and data types as core constructs useful for data science applications, but their use is otherwise uninteresting – the ideas in this paper are readily applicable to a language using a more traditional pattern matching mechanism.)

A variable eliminator $x \mapsto \kappa$ says how to match any value (as variable x) and continue as κ . A record eliminator $\{\vec{x}\} \mapsto \kappa$ says how to match a record with fields $\vec{x}$, and provides a continuation κ for matching the values of those fields. Lastly, the eliminator $\{c \mapsto \kappa\}$ provides a branch $c \mapsto \kappa$ for each constructor c in $\vec{c}$, where each κ may specify how the arguments to c will be matched. (We assume that any constructors matched by an eliminator all belong to the same data type, but this is not enforced in the core language.)

4.1.4 Values and environments. We define addressed *values* v mutually inductively with *raw values* ν , a raw value does not have an associated address. These addresses, explained in § 4.1.5, will (later) form the vertices of the graphs that we build in the graph semantics.

Raw values ν include integers n, m ; records $\{\vec{x} \mapsto \vec{\nu}\}$; and constructor values $c(\vec{\nu})$. We also define function closures as $\text{cl}(\gamma, \rho, \sigma)$ where σ is the function (eliminator), γ is its environment, and to support mutual recursion, ρ is the (possibly empty) environment of functions (eliminators) with which σ was mutually defined. An environment is defined as a finite map. In our setting, IO analysis of programs, we are often only interested in some strict subset of the environment $\Gamma' \subseteq \Gamma$. Happily, domain restriction of maps induces a pair of conjugate functions: one of them maps the unselected part of the environment $(\Gamma \setminus \Gamma')$ to $\top$, and the other maps the unselected values to $\perp$. In practical terms, this mechanism could be used to allow for “focussed” queries – the ability to restrict the parts of the output or input that the user is interested in using to mediate the linked inputs or outputs query.

4.1.5 Addresses for partial values. The addresses α that annotate values act as identifiers, uniquely corresponding those values to specific vertices in a dependence graph. There are many sensible ways of assigning addresses (our particular implementation uses an integer counter to generate

$$\begin{array}{c}
\boxed{\vec{v}, \kappa \rightsquigarrow \gamma, e, V} \\
\\
\begin{array}{cc}
\rightsquigarrow\text{-done} & \rightsquigarrow\text{-var} \\
\frac{}{\epsilon, e \rightsquigarrow \emptyset, e, \emptyset} & \frac{\vec{v}, \kappa \rightsquigarrow \gamma, e, V}{v \cdot \vec{v}, x \mapsto \kappa \rightsquigarrow \gamma \cdot (x : v), e, V} \\
\\
\rightsquigarrow\text{-record} & \rightsquigarrow\text{-constr} \\
\frac{\{\vec{y} : \vec{u}\} \subseteq \{\vec{x} : \vec{v}\} \quad \vec{u} ++ \vec{v}', \kappa \rightsquigarrow \gamma, e, V}{\{\vec{x} : \vec{v}\}_\alpha \cdot \vec{v}', \{\vec{y}\} \mapsto \kappa \rightsquigarrow \gamma, e, \alpha \cdot V} & \frac{\vec{v} ++ \vec{v}', \kappa \rightsquigarrow \gamma, e, V}{c(\vec{v})_\alpha \cdot \vec{v}', (c \mapsto \kappa) \cdot \{\vec{c} \mapsto \vec{k}\} \rightsquigarrow \gamma, e, \alpha \cdot V} \Sigma(c) = |\vec{v}|
\end{array}
\end{array}$$

Fig. 10. Pattern-matching

$$\begin{array}{c}
\boxed{\gamma, e, V \Rightarrow v, G} \\
\\
\begin{array}{ccc}
\Rightarrow\text{-var} & \Rightarrow\text{-int} & \Rightarrow\text{-function} \\
\frac{}{\gamma \cdot (x : v), x, V \Rightarrow v, \emptyset} & \frac{}{\gamma, n, V \Rightarrow n_\alpha, \{V \leftarrow \alpha\}} & \frac{}{\gamma, \lambda \sigma, V \Rightarrow \text{cl}(\gamma, \emptyset, \sigma)_\alpha, \{V \leftarrow \alpha\}} \\
\\
\Rightarrow\text{-record} & \Rightarrow\text{-constr} \\
\frac{\gamma, \vec{e}, V \Rightarrow \vec{v}, G}{\gamma, \{\vec{x} : \vec{e}\}, V \Rightarrow \{\vec{x} : \vec{v}\}_\alpha, G \uplus \{V \leftarrow \alpha\}} & \frac{\gamma, \vec{e}, V \Rightarrow \vec{v}, G}{\gamma, c(\vec{e}), V \Rightarrow c(\vec{v})_\alpha, G \uplus \{V \leftarrow \alpha\}} \Sigma(c) = |\vec{e}| \\
\\
\Rightarrow\text{-record-project} & \Rightarrow\text{-foreign-app} \\
\frac{\gamma, e, V \Rightarrow \{\vec{x} : \vec{v} \cdot (y : u)\}_\alpha, G}{\gamma, e.y, V \Rightarrow u, G} & \frac{\gamma, \vec{e}, V \Rightarrow \vec{v}, G \quad \hat{f}(\vec{v}) = (u, G')}{\gamma, f(\vec{e}), V \Rightarrow u, G \uplus G'} \Phi(f) = |\vec{e}| \\
\\
\Rightarrow\text{-let} & \Rightarrow\text{-let-rec} \\
\frac{\gamma, e, V \Rightarrow v, G \quad \gamma \cdot (x : v), e', V \Rightarrow v', G'}{\gamma, \text{let } x = e \text{ in } e', V \Rightarrow v', G \uplus G'} & \frac{\gamma, \{\vec{x} : \vec{\sigma}\}, V \mapsto \gamma', G \quad \gamma ++ \gamma', e, V \Rightarrow v, G'}{\gamma, \text{let } \vec{x} : \vec{\sigma} \text{ in } e, V \Rightarrow v, G \uplus G'} \\
\\
\Rightarrow\text{-app} \\
\frac{\gamma, e, V \Rightarrow \text{cl}(\gamma_1, \rho, \sigma)_\alpha, G_1 \quad \gamma_1, \rho, \{\alpha\} \mapsto \gamma_2, G_2 \quad \gamma, e', V \Rightarrow v', G_3 \quad v', \sigma \rightsquigarrow \gamma_3, e'', V' \quad \gamma_1 ++ \gamma_2 ++ \gamma_3, e'', V' \uplus \{\alpha\} \Rightarrow u, G_4}{\gamma, e e', V \Rightarrow u, \uplus \{G_1, \dots, G_4\}}
\end{array}
\end{array}$$

$$\begin{array}{c}
\boxed{\gamma, \vec{e}, V \Rightarrow \vec{v}, G} \\
\\
\Rightarrow\text{-seq} \\
\frac{\gamma, e_i, V \Rightarrow v_i, G_i \quad (\forall i \leq |\vec{e}|)}{\gamma, \vec{e}, V \Rightarrow \vec{v}, \uplus \vec{G}}
\end{array}$$

Fig. 11. Operational semantics: evaluation and graph construction

new ones) as long as some freshness condition is satisfied – ensuring that there are no unwanted address collisions in a given value i.e. graph. For simplicity, we elide this side-condition in the graph semantics (next) and assume it is met whenever a new address α is introduced.

4.2 Evaluation and Graph Construction

4.2.1 Pattern matching. Figure 10 defines the pattern-matching judgement $\vec{v}, \kappa \rightsquigarrow \gamma, e, V$. Rather than matching a single value v against an eliminator σ , the judgement matches a “stack” of values

$$\gamma, \rho, V \rightsquigarrow \gamma', G$$

with $\text{dom}(\gamma') = \text{dom}(\rho)$

$$\frac{v_i = \text{cl}(\gamma, \rho, \rho(x_i))_{\alpha_i} \quad G_i = \{\alpha_i : V\} \quad (\forall i \leq |\vec{x}|)}{\gamma, \rho, V \rightsquigarrow \{\vec{x} : \vec{v}\}, \uplus \vec{G}}$$

Fig. 12. “Closing over” recursive definitions

$\vec{v}$ against a continuation κ . It then returns the selected branch e , an environment γ providing bindings for the free variables in e , and a set V of vertices found in the matched portions of $\vec{v}$.

The $\rightsquigarrow$ -done rule expresses exactly when using a *term* e as the continuation may succeed: when the stack of values to be matched is empty, and so e is treated as the selected term to be executed. The other rules require an *eliminator* σ as the continuation – using this to match the head of a non-empty stack $v \cdot \vec{v}$, unpack any of its relevant subvalues, and then push them onto the tail $\vec{v}$ to be recursively matched by the continuation selected in σ .

A variable eliminator $x \mapsto \kappa$ matches any value v and returns no vertices, reflecting the fact that no part of v would need to be consumed to select κ as the continuation. The eliminator $\{\vec{y}\} \mapsto \kappa$ only matches with a record $\{\vec{x} : \vec{v}\}_\alpha$ if the variables in $\vec{y}$ are also fields in $\vec{x}$, returning the vertex α associated with the record; this condition is specified by the premise $\{\vec{y} : \vec{u}\} \subseteq \{\vec{x} : \vec{v}\}$, which simultaneously projects out the corresponding values of $\vec{u}$ from $\vec{v}$. Lastly, an eliminator with a branch ($c \mapsto \kappa$) only matches a value $c(\vec{v})_\alpha$ constructed by c , and returns the vertex α associated with that particular instance of c .

4.2.2 Evaluation. Figure 11 defines a big-step evaluation relation $\gamma, e, V \Rightarrow v, G$ for the core language, stating that term e , under an environment γ and vertex set V , evaluates to a value v plus dependence graph G . The input set V corresponds to the inputs consumed by the current active function call from which e is being evaluated, and can be thought of as the “ambient” set of dependencies; this is initially empty for a top-level term, and may grow whenever a function is applied to an argument and pattern-matching occurs (i.e. in the $\Rightarrow$ -app rule, which we explain later). The output graph G is built by specifying new graph fragments and then combining them; for convenience, we write $\{V \leftarrow \alpha\}$ as shorthand for the star graph $(\{\alpha\} \cup V, V \times \{\alpha\})$ indicating that vertex α depends on all vertices in V .

The $\Rightarrow$ -var rule for evaluating variables is standard, and the returned dependence graph is empty; that is, no new vertices are constructed as a result of simply looking up values. Most of the other evaluation rules can generally be understood in terms of two recurring patterns.

First, for any introduction rule that creates a new (partial) value, there is no choice but to allocate that value with a new vertex (address) which depends on ambient vertices V . These cases include $\Rightarrow$ -int, $\Rightarrow$ -function, $\Rightarrow$ -record, and $\Rightarrow$ -constr. For example, $\Rightarrow$ -int evaluates an integer n to its value form n_α assigned a fresh vertex α ; the resulting dependence graph is then $\{\alpha : V\}$, denoting that evaluating to n_α depended on any inputs that were found in V . Likewise, $\Rightarrow$ -function evaluates an anonymous function $\lambda\sigma$ by constructing the closure $\text{cl}(\gamma, \emptyset, \sigma)_\alpha$ annotated by a new vertex α ; here, the closure captures the current environment γ and uses $\emptyset$ as the set of mutually recursive definitions. The “exception” to this pattern is $\Rightarrow$ -record-project for $e.y$, which evaluates a record e to $\{\vec{x} : \vec{v} \cdot (y : u)\}_\alpha$ before returning the value u associated to field y ; while this indeed introduces a record value allocated with a new vertex α , the record as a whole goes unused, and so α is omitted in the final dependence graph.

The second recurring pattern is that, for any rule which involves evaluating subterms (of a term), the corresponding subgraphs that are constructed must be disjoint – reflecting that there is no overlap between their vertex addresses – and their union is the dependence graph of the

<pre> 1 let filter p [] = []; 2 filter p (x : xs) = 3 let ys = filter p xs in 4 if p x then x : ys else ys 5 in 6 filter ((<) 5) [8, 4, 7, 3] </pre>	<pre> 1 let xs = [[1, 4, 8], 2 [3, 2, 17], 3 [0, 14, 6]] 4 in [nth2 i j xs (i, j) in (3, 3)] </pre>
(a) Piecewise functions	(b) Matrix syntax
<pre> 1 [x + y x <- [5, 4, 3], y <- [9, 7, 5]] </pre>	<pre> 1 let d = { "a" := 5, "a" ++ "b" := {fst: 6, snd: 0} } 2 in dict_get "a" d * (dict_get "ab" d).snd </pre>
(c) List comprehensions	(d) Dictionaries and records

Fig. 13. Some example programs in our surface language

overall term. This general principle is captured by the auxiliary evaluation relation $\gamma, \vec{e}, V \Rightarrow \vec{v}, G$ (bottom of Figure 11), which is for evaluating a sequence of terms $\vec{e}$. Its single rule evaluates each term $e_i \in \vec{e}$ to a value $v_i \in \vec{v}$ and graph $G_i \in \vec{G}$, and builds the dependence graph $\biguplus \vec{G}$. We make use of this relation in the rules $\Rightarrow$ -record for $\{\vec{x} : \vec{e}\}$, $\Rightarrow$ -constr for $c(\vec{e})$, and $\Rightarrow$ -foreign-app for $f(\vec{e})$, which all involve evaluating a sequence $\vec{e}$ of subterms; in the last rule, $\hat{f}$ is the foreign implementation that provides a way to evaluate f to a graph and value (as required in § 4.1.2). Of course, this pattern applies to subterms in general and not just those expressed as a sequence $\vec{e}$. For example, the $\Rightarrow$ -let rule for $\text{let } x = e \text{ in } e'$ (which otherwise behaves as standard) simply unions the graphs of subterms e and e' , and similarly for $\Rightarrow$ -let-rec and $\Rightarrow$ -app.

To fully explain the last two rules, $\Rightarrow$ -let-rec and $\Rightarrow$ -app, we first detail how our language deals with *mutual recursion*. For this, Figure 12 defines the auxiliary relation $\gamma, \rho, V \mapsto \gamma', G$, which takes an environment ρ of recursive definitions and evaluates it to a value environment γ' of closures, while also building a dependence graph G . This works by turning each function definition $\rho(x_i) \in \rho$ into a newly allocated closure $(\gamma, \rho, \rho(x_i))_{\alpha_i}$ as a fresh vertex α_i , capturing ρ and a copy of the original environment γ , where each closure depends on ambient inputs V (like in evaluation). The final dependence graph is the combination of each closure's subgraph (also like in evaluation).

Returning to Figure 11, rule $\Rightarrow$ -let-rec for $\text{let } \vec{x} : \vec{\sigma} \text{ in } e$ is then the same as regular let-bindings, except using $\mapsto$ to evaluate recursive definitions $\vec{x} : \vec{\sigma}$. The final rule, $\Rightarrow$ -app for $e e'$, is especially important to the graph semantics, as it is during application that we grow the ambient inputs V that a given evaluation step depends on. Here, we compute the closure $\text{cl}(\gamma_1, \rho, \sigma)_{\alpha}$ of function e , and use its eliminator σ to pattern-match against the value v' of argument e' . What is returned includes (i) the *selected branch* e'' and (ii) the vertex set V' , where V' represents the values in v' consumed by pattern-matching in order to lead to e'' . The last premise which executes the branch then specifies its ambient dependencies as $V' \uplus \{\alpha\}$; that is, the (closure of the) function that was applied, α , and the (consumed values of the) argument it was applied to, V' .

4.2.3 Surface Language. Our implementation provides a surface language which desugars into the core language presented above; in particular the surface language supports piecewise function definitions (which desugar into eliminators) and conventional list comprehensions. The core and surface languages also both support matrices, using a comprehension-like syntax for matrix initialisation. The examples in Figure 13 illustrate some of these features; for more complex examples, we refer the reader to the listings in the supplementary material.

5 EVALUATION

We now present a critical evaluation of our framework, comparing with the popular *trace-based* approach to bidirectional program analysis [Ricciotti et al. 2017; Psallidas and Wu 2018]. In particular,

we contrast with the state of the art in the field by Perera et al. [2022]. We start by considering our framework’s qualitative advantage in terms of conceptual simplicity and ease of implementation. Then, we provide an experimental comparison of our implementation’s performance.

5.1 Qualitative Evaluation of Our Approach

Here, we consider the relative implementation burden of our graph approach compared to the trace approach in Perera et al. [2022], hypothesising that our graph one is easier to implement and maintain.

Firstly, a key benefit of the graph approach is that it avoids the need to implement a backwards evaluation function. By contrast, the trace approach taken by Perera et al. [2022] (and others) use a bidirectional interpreter: building an execution *trace* during forward execution, and then rewinding that trace to perform a backwards dependency analysis such as ∇_R . Hence, the developer has to implement a separate backwards evaluation semantics using the execution trace, which propagates a selected output back to the program inputs. This approach is burdensome. The two directions of the interpreter need to be kept synchronised with respect to the dependency analysis, which makes updating the language difficult as every change requires modifying each direction in a manner that maintains bidirectionality. Conversely, in our approach, once the dependence graph has been computed, the algorithm described in Figure 5a is used to compute both ∇_R and Δ_R over the graph.

Secondly, our algorithm for computing ∇_R and Δ_R is language-agnostic (as described in § 3), meaning that it is not affected by changes in the semantics of the language. This is as opposed to works like Perera et al. [2022], where analysis is implemented as part of (forward and backward) program evaluation, and hence heavily depends on the details of the language. Our analysis is run over a graph that is not language dependent – it only encodes the *values produced* by (forward) program evaluation.

5.2 Experimental Evaluation

In order to test the performance of our approach, we implemented our language from § 4 on top of PureScript, which we used to write a selection of non-trivial examples that reflect realistic targets of dependency analyses (§ 5.2.1). PureScript is a natural implementation language for this work as our focus on visualisations lends itself to building web-based interfaces. Additionally, PureScript is the implementation language used by Perera et al. [2022] in their open-source implementation of the trace-based approach, thus enabling a more fair like-for-like comparison to our work here. We then benchmarked the implementations on an Intel Core i7-10850H 2.70GHz with 16Gb of RAM, using Google Chrome 121.0.6261.69 and JavaScript runtime V8 12.2.281.16. We instrumented both systems in order to extract performance numbers.

In an interactive system, performance plays a critical role in a user’s experience. We compare the performance of our graph-based approach with the trace-based [Perera et al. 2022] by considering the following three research questions:

- RQ1: What is the overhead of the graph construction versus the trace construction?
- RQ2: How does the performance of graph-based *demands* compare against the performance of trace-based *demands*?
- RQ3: How does the performance of the various implementations of *demanded by* (i.e. trace-based, graph-based using demands, and graph-based using the dual of suff) compare?

We judge performance according to the performance standards proposed in Nielsen [1994]:

- **100ms**: this is the limit for an interaction feeling *instantaneous*
- **1000ms**: this is the limit for a user’s train of thought to remain uninterrupted, albeit they will notice a delay

- **10000ms**: this is the limit to keep a user's attention on the system

We aim for the interactive queries to stay as close as possible to the *instantaneous* category (captured by RQ2 and RQ3), and for the time taken to evaluate programs (corresponding to RQ1) to be within the limit for keeping a user's attention.

5.2.1 Choice of examples. We tested our system on a range of benchmark examples implemented in [OurLanguage] and in the language of Perera et al. [2022]. The benchmark programs represent a range of components a data scientist may find useful in an exploratory data analysis scenario. First, we considered matrix convolution (**edge-detect**, **emboss**, **gaussian**, 31 Lines of Code (LoC) each). Given that different filter/kernel matrices create subtly different dependency structures, we implemented a few different such kernels. Second, we developed a prototype of a graphics library that demonstrates the performance on bespoke visualisation code (**grouped-bar-chart**, 140 LoC, **line-chart**, 143 LoC, **stacked-bar-chart**, 136 LoC). Finally, we test two linked examples (**stacked-bar-scatter-plot** (corresponding to Figure 3 from § 2), 26 LoC, and **bar-chart-line-chart**, 38 LoC). The code for our benchmarks is given in the supplementary material.

In all our experiments, each benchmark is run 10 times, and we report the mean runtime and standard deviation (the latter is shown in the results tables coloured gray in brackets). Runtimes are reported in milliseconds (to 2 decimal places). When we refer to *speedup* or *slowdown* in reference to a pair of implementations, we mean the ratio of the runtimes for a given benchmark.

5.2.2 RQ1: Overhead of graph construction vs. trace construction. In Table 1, we compare the time taken to evaluate a program in the graph approach (**G-Eval**) of this paper with the time taken to evaluate a program with the trace approach (**T-Eval**) of Perera et al. [2022]. As previously explained, the graph and trace are built as part of the respective program evaluations. Since the graph is a more complex data structure to build compared to the trace, we expect a larger overhead when evaluating a program to a graph compared to evaluating it to a trace.

We report the ratio of graph evaluation time to trace evaluation time in the third column (**Eval-Slowdown**) of Table 1. We see that, as expected, the evaluation of programs to their dynamic dependence graph does indeed incur a higher overhead, being from 3x to 14x slower than their trace-based counterparts. For both approaches, program evaluation always stays within the limit to keep a user's attention on the system.

Table 1. Evaluation Time of Traces Versus Graphs (ms, 2 d.p.)

	T-Eval	G-Eval	Eval-Slowdown
edge-detect	732.38 (± 44.73)	2150.13 (± 109.85)	2.94
emboss	633.36 (± 42.31)	1718.45 (± 30.93)	2.71
gaussian	619.93 (± 48.71)	1714.26 (± 42.52)	2.77
bar-chart-line-chart	2336.85 (± 51.86)	5775.65 (± 131.59)	2.47
stacked-bar-scatter-plot	1212.89 (± 46.21)	7548.93 (± 476.35)	6.22
grouped-bar-chart	89.25 (± 8.36)	1341.60 (± 132.35)	15.03
line-chart	130.02 (± 10.53)	1131.00 (± 14.42)	8.70
stacked-bar-chart	55.92 (± 5.61)	783.03 (± 15.42)	14.00

5.2.3 RQ2: Graph-based demands vs. trace-based demands. In Table 2, columns **T-Demands** and **G-Demands** are the times taken to compute *demands* with the trace and graph approaches, respectively. The ratio of graph-based to trace-based performance is in the last column, **Bwd-Speedup**.

Overall, we observe a significant speedup in the performance of *demands* using the graph approach, from 33x speedup for the **stacked-bar-chart**, to more than 80x for **gaussian**. In particular, every *demands* query using the graph based approach is within the “instantaneous” category, whilst all but two of the examples using the trace approach are in the “noticable delay” category. We attribute this performance improvement to the fact that, in the trace approach, each new query requires the system to rewind the *entire* execution of the original program, in effect evaluating it in reverse. This is regardless of whether all of that trace is relevant. By contrast, in the current graph approach, one only needs to traverse the relevant part of the already-computed graph.

Table 2. Demands Times of Traces Versus Graphs (ms, 2 d.p.)

	T-Demands	G-Demands	Bwd-Speedup
edge-detect	604.96 (± 7.9)	11.90 (± 0.76)	50.84
emboss	504.33 (± 9.12)	7.97 (± 0.66)	63.28
gaussian	511.06 (± 8.33)	6.28 (± 0.77)	81.38
bar-chart-line-chart	1273.98 (± 24.78)	18.50 (± 0.57)	68.86
stacked-bar-scatter-plot	616.63 (± 35.39)	13.97 (± 1.7)	44.14
grouped-bar-chart	74.43 (± 6.22)	1.33 (± 0.27)	55.96
line-chart	114.13 (± 4.84)	1.75 (± 0.21)	65.22
stacked-bar-chart	45.05 (± 5.38)	1.34 (± 0.35)	33.62

5.2.4 RQ3: Implementations of Demanded By. In Table 3, we investigate the performance of *demanded by* in different settings: column **T-DemBy** provides the results for the trace implementation, column **G-DemBy** contains the results for the graph approach using the demBy algorithm in Figure 5a, and column **G-DemBy-Suff** contains the results obtained when computing *demanded by* by taking the De Morgan dual of suff from Figure 5b. Additionally, in column **Speedup**, we provide the speedup of **G-DemBy** compared to **T-DemBy**, and in column **Suff-Speedup**, we give the speedup of **G-DemBy-Suff** compared to **T-DemBy**.

For all benchmarks, we observe that the graph approach using the demBy algorithm in Figure 5a performs better than the other two approaches, with five of the graph based benchmarks running instantaneously, as opposed to only two for the trace based approach.

Table 3. Equivalent implementations of Demanded By

	T-DemBy	G-DemBy	Speedup	G-DemBy-Suff	Suff-Speedup
edge-detect	696.90 (± 14.85)	265.81 (± 19.08)	2.62	664.32 (± 36.31)	1.05
emboss	603.04 (± 18.03)	244.93 (± 16.31)	2.46	481.51 (± 16.38)	1.25
gaussian	596.78 (± 11.59)	224.27 (± 17.43)	2.66	474.01 (± 11.96)	1.26
bar-chart-line-chart	2078.98 (± 97.24)	19.12 (± 1.23)	108.73	2360.16 (± 47.96)	0.88
stacked-bar-scatter-plot	1004.09 (± 72.66)	31.31 (± 1.33)	32.07	11598.93 (± 778.78)	0.09
grouped-bar-chart	82.37 (± 9.43)	5.00 (± 0.58)	16.47	2146.46 (± 157.97)	0.04
line-chart	113.43 (± 4.87)	2.25 (± 0.52)	50.41	900.31 (± 15.26)	0.13
stacked-bar-chart	42.56 (± 1.85)	3.23 (± 0.97)	13.18	1271.05 (± 131.68)	0.03

5.2.5 Discussion of the results. When comparing the performance of the graph and trace approaches, we observe that the only consistent performance loss for the graph approach is due to the increased

cost of evaluating a program to a graph versus evaluating it to a trace (according to *RQ1*). We argue that this overhead is reasonable: the graph only needs to be constructed once and does not impact subsequent interactions; and program evaluation time does not exceed the limit for keeping the users attention on the system. Additionally, the fast response times for *demands* and *demanded by* (according to *RQ2* and *RQ3*), compared to the trace approach, translate into quick responses to user queries. This is critical for the interactive exploration of digital media.

Another notable aspect, which is observed for *RQ3*, is the performance of **G-DemBy-Suff**, which takes the dual of the suff algorithm. In the majority of cases, **G-DemBy-Suff** performs significantly worse than **G-DemBy**. We suspect that the generally bad performance of **G-DemBy-Suff** is at least partially caused by the expensive set complement operations performed before and after applying suff. Furthermore, while our algorithm for computing *demanded by* is graph independent (as described in § 3), these experiments show that the shape of the graph may matter for performance. We defer more detailed discussion to § 7.1.

6 RELATED WORK

6.1 Applying Dynamic Dependence Graphs

We are not the first to consider using dynamic dependence graphs for functional programming languages. For example, the compilation of a functional program to a graph has been used to implement incremental evaluation of programs in [Acar et al. \[2002, 2006\]](#). In contrast to their work, which focusses on efficiently propagating changes for inputs to computations, we are concerned with the implementation of interactive data exploration systems.

For a more generic perspective on program slicing, [Field and Tip \[1998\]](#) consider dynamic dependence tracking in term-rewriting systems. Their approach allows them to perform graph-based program slicing on a subset of Pascal, but differs from our work in two ways. Firstly, they are concerned primarily with program slicing of imperative programs as opposed to building interactive visualisations, and secondly, they explicitly represent reduction steps in their dependence graphs. In contrast, we do not represent this sort of information in the graph at all, and instead of representing the semantics as a term-rewriting system, we present ours as an extended big-step relation.

More recently there has been a body of work aimed at using dynamic dependence graphs to assist in program fault localisation, but again these are not focussed on interactive visualisation systems, but rather on bug finding in programs. For a survey see [Soremekun et al. \[2021\]](#).

6.2 Galois Slicing

Galois connections between lattices offer an appealing mathematical foundation for bidirectional dynamic program analysis, cleanly characterising input-output relationships in terms of demand and sufficiency [\[Perera et al. 2012, 2016; Ricciotti et al. 2017\]](#). In these works, Galois connections are applied to the problem of program slicing. The upper adjoint – which characterises forward slicing from inputs to outputs – can be thought of as a *sufficiency* operator, picking out all the outputs which can be computed using only a particular set of inputs; the lower adjoint, which characterises backward slicing, is a *demand* operator, picking out the specific inputs needed to compute a particular set of outputs. [Perera et al. \[2022\]](#) extended this approach to a setting where slices have complements and the slices of a particular program or value form a Boolean lattice; they then showed how the De Morgan dual of the upper adjoint can be used to pick out *linked outputs*, namely the other outputs that “compete” with a given set of outputs in the sense of demanding some of the same inputs, and proposed this as a framework for understanding the “brushing and linking” feature commonly used in data visualisation.

However, implementations of Galois slicing efficient enough to support the kind of interactive applications considered here have proved elusive. To compute the two analyses, implementations have relied on a sequential execution record, usually taking the form of a big-step or small-step trace. The forward analysis uses the trace to “replay” the computation, propagating information about resource availability from inputs to outputs; the backwards analysis uses the trace to “rewind” the computation, propagating information about demand from outputs to inputs. This overly sequential approach makes the approach unable to exploit the very independence that makes slicing useful in the first place. Moreover, the backwards analysis makes frequent use of lattice join ($\vee$) to combine slicing information computed along different branches of the computation. Joining closure slices in particular becomes expensive because of the contained environments (which in turn contain closures), and these issues have made the approach prohibitively slow for interactive use. Compared to Galois slicing, our approach of decomposing the problem into a graph semantics and a pair of analyses over the graph significantly reduces the implementation burden on the developer, since the algorithms for the program analyses are language-agnostic and thus only need to be implemented once.

6.3 Interactive Visualisation

We are not the first to consider shifting the burden of interactive visualisations into the language runtime. Reactive Vega [Satyanarayan et al. 2017] presents a system for building interactivity in visualisations. Vega is a declarative DSL for constructing interactions with visualisations. The features of Vega allow for more complicated interactions than our own work at present, however these are hampered by requiring the designer to learn multiple separate languages. Further, the syntax of Vega for combining these interactions is non-obvious to a new user. In contrast, whilst our work provides a smaller set of interaction primitives it frees up the developer to focus on building their data analysis programs, leaving the burden of implementing selections to the language itself. The benefit of including everything into the base language is that it reduces the number of tools that a developer needs proficiency with.

Our linking of visualisations is similar in spirit to the work of North and Shneiderman [2000]. In that work, the authors design a modular system for replacing visualisations of databases. Their system is modelled by relational database schema, and is designed to facilitate interactions with these modular visualisations, but is limited to data from relational databases, whereas ours can be applied to data of any shape, as long as it is supported by our language. Devise [Livny et al. 1997] is another visualisation framework for building interactive visualisations of relational database tables. Similar to North and Shneiderman [2000], their work is defined by its usage of relational databases, and the way they create a semantics for queries on visualisations using the model of relational database queries.

More recent database provenance work Dietrich et al. [2022] attempts to simultaneously explain both value and control dependencies in recursive queries and user-defined functions. Their work takes an SQL query and rewrites it into a pair of queries which compute not only the output table, but the provenance of the values it contains. In order to modify our approach to work in this setting, we would need to encode extra information into the edges of the graph, specifically whether an edge is a control-flow edge – coming from an if-else or a pattern match – or a value edge – from producing a new value out of old ones. This information roughly corresponds to the “why” and “where” provenance described in the database setting. Further we would need to use a dependency relation which agrees with the semantics of an SQL query or user-defined functions, since a relational database is semantically distinct from our functional setting.

7 CONCLUSION AND FUTURE WORK

Our work is motivated by the difficulty and time-consuming nature of producing interactive data visualisations. Visualisations provide a valuable mechanism for communicating research and performing exploratory data analysis, but interactive visualisations are often bespoke solutions for a single application. By shifting the burden into a language runtime, time and effort can be saved by the developer, who needs only to express their data analysis as a program creating the relevant charts, and gets interaction functionality for free. However, prior systems that aimed at integrating these features proved to be sensitive to changes in the programming language, and made use of bidirectional analyses which were difficult to implement.

In this paper, we introduced a novel bidirectional program analysis framework that decomposes the system into an evaluator that builds a dynamic dependence graph, and a pair of language agnostic graph algorithms. By doing so, we achieve a separation of the analyses from the language, a simpler implementation of the analyses themselves, and an overall improved performance for running analyses.

7.1 Future work

In order to enable a like-for-like comparison, the dependency relation we chose to represent in the graph is the same as that of [Perera et al. \[2022\]](#); here, selections are pushed through evaluation if the selected parts of values are consumed during evaluation - either via pattern matching or the specific implementation provided for a primitive. However, the dependency relation used in that paper was not designed for application in a graph-based setting.

Our results in § 5 raise the question of how important the structure of the graph is in the performance of our analysis, and whether our framework would benefit from an alternative representation. For example, rather than representing all dependencies as value-to-value dependencies, it may be more useful for edges in the graph to represent causal relationships between steps in the program evaluation. We suggest that a more semantically justified dependency relation could lead to improvements in the scalability of the system, and allow for extended functionality to be implemented.

The analyses in their current form are only sensitive to the *extension* – the I/O relation – of the graph. Two graphs which are *intensionally* different will produce the same conjugate operators if their I/O relation is the same. Using the intensional information present within a graph may allow us to derive more detailed information about the relationships between inputs and outputs. As an example, [Cheney et al. \[2011\]](#) considers the use of dependency analysis to compute *expression provenance* information about elements of a program. One use of this information is to synthesize a sub-expression describing the derivation of a value at a given point. From the perspective of the graph approach, we suggest that access to intensional information could allow us to take a graph slice and use it to synthesize a subexpression of the original program which can be executed to produce a subset of the outputs.

A final point of inquiry is raised by the relative difference in performance between the implementations of *demanded by*. It would be interesting to investigate if there are structural properties of the dependence graph that determine the efficiency of the implementations of *demanded by*. If these structural properties exist, we would like to know how they can be computed, and if they can be computed efficiently. It follows – if the user is entirely uninterested in the intensional information of their graph – that transformations which elide internal nodes but preserve I/O connectivity, such as the classical Y- Δ transform [\[Truemper 1989\]](#), will produce an extensionally equivalent (but faster) analysis. We conjecture that applying this transform to throw away intensional information may be useful in future applications of our work.

REFERENCES

- Umut A. Acar, Guy E. Blelloch, and Robert Harper. 2002. Adaptive functional programming. In *POPL '02: Proceedings of the 29th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Portland, Oregon). ACM Press, New York, NY, USA, 247–259. <https://doi.org/10.1145/503272.503296>
- Umut A. Acar, Guy E. Blelloch, and Robert Harper. 2006. Adaptive Functional Programming. *ACM Trans. Program. Lang. Syst.* 28, 6 (nov 2006), 990–1034. <https://doi.org/10.1145/1186632.1186634>
- Hiralal Agrawal and Joseph R. Horgan. 1990. Dynamic program slicing. In *PLDI '90: Proceedings of the ACM SIGPLAN 1990 Conference on Programming Language Design and Implementation* (White Plains, New York, United States). ACM, New York, NY, USA, 246–256. <https://doi.org/10.1145/93542.93576>
- Nadieh Bremer and Marlieke Ranzijn. 2015. Urbanization in East Asia between 2000 and 2010. <http://nbremer.github.io/urbanization/>.
- James Cheney, Amal Ahmed, and Umut A. Acar. 2011. Provenance as dependency analysis. *Mathematical Structures in Computer Science* 21, 6 (2011), 1301–1337.
- R. DerSimonian and N. Laird. 1986. Meta-analysis in clinical trials. *Control Clin Trials* 7, 3 (Sep 1986), 177–88. [https://doi.org/10.1016/0197-2456\(86\)90046-2](https://doi.org/10.1016/0197-2456(86)90046-2)
- Benjamin Dietrich, Tobias Müller, and Torsten Grust. 2022. Data provenance for recursive SQL queries. In *Proceedings of the 14th International Workshop on the Theory and Practice of Provenance*. 1–8.
- Jeanne Ferrante, Karl J. Ottenstein, and Joe D. Warren. 1987. The Program Dependence Graph and Its Use in Optimization. *ACM Trans. Program. Lang. Syst.* 9, 3 (jul 1987), 319–349. <https://doi.org/10.1145/24039.24041>
- John Field and Frank Tip. 1998. Dynamic Dependence in Term Rewriting Systems and its Application to Program Slicing. *Information and Software Technology* 40, 11–12 (November/December 1998), 609–636.
- Ralf Hinze. 2000. Generalizing generalized tries. *Journal of Functional Programming* 10, 4 (2000), 327–351. <https://doi.org/10.1017/S0956796800003713>
- Bjarni Jonsson and Alfred Tarski. 1951. Boolean Algebras with Operators. Part I. *American Journal of Mathematics* 73, 4 (1951), 891–939. <http://www.jstor.org/stable/2372123>
- M. Livny, R. Ramakrishnan, K. Beyer, G. Chen, D. Donjerkovic, S. Lawande, J. Myllymaki, and K. Wenger. 1997. DEVise: integrated querying and visual exploration of large datasets. *SIGMOD Rec.* 26, 2 (jun 1997), 301–312. <https://doi.org/10.1145/253262.253335>
- M. Menni and C. Smith. 2014. Modes of Adjointness. *Journal of Philosophical Logic* 43, 2/3 (2014), 365–391. <http://www.jstor.org/stable/24564097>
- James Murphy, David Sexton, David Barnett, Gareth Jones, M. Webb, M. Collins, and David Stainforth. 2004. Quantification of modelling uncertainties in a large ensemble of climate change simulations. *Nature* 430 (09 2004), 768–72. <https://doi.org/10.1038/nature02771>
- Jakob Nielsen. 1994. *Usability Engineering*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA.
- Chris North and Ben Shneiderman. 2000. Snap-together visualization: a user interface for coordinating visualizations via relational schemata. In *Proceedings of the Working Conference on Advanced Visual Interfaces* (Palermo, Italy) (AVI '00). Association for Computing Machinery, New York, NY, USA, 128–135. <https://doi.org/10.1145/345513.345282>
- Roly Perera, Umut A. Acar, James Cheney, and Paul Blain Levy. 2012. Functional Programs That Explain Their Work. In *Proceedings of the 17th ACM SIGPLAN International Conference on Functional Programming* (Copenhagen, Denmark) (ICFP '12). ACM, New York, NY, USA, 365–376. <https://doi.org/10.1145/2364527.2364579>
- Roly Perera, Deepak Garg, and James Cheney. 2016. Causally Consistent Dynamic Slicing. In *Concurrency Theory, 27th International Conference, CONCUR '16 (Leibniz International Proceedings in Informatics (LIPIcs))*, Josée Desharnais and Radha Jagadeesan (Eds.). Schloss Dagstuhl–Leibniz-Zentrum für Informatik, Dagstuhl, Germany. <https://doi.org/10.4230/LIPIcs.CONCUR.2016.18>
- Roly Perera, Minh Nguyen, Tomas Petricek, and Meng Wang. 2022. Linked Visualisations via Galois Dependencies. *Proc. ACM Program. Lang.* 6, POPL, Article 7 (2022), 29 pages. <https://doi.org/10.1145/3498668>
- Simon Peyton Jones, Richard Eisenberg, and Sebastian Graf. 2022. *Triemaps that match*. Technical Report. <https://simon.peytonjones.org/triemaps-that-match/>
- Fotis Psallidas and Eugene Wu. 2018. Smoke: fine-grained lineage at interactive speed. *Proc. VLDB Endow.* 11, 6 (feb 2018), 719–732. <https://doi.org/10.14778/3199517.3199522>
- Wilmer Ricciotti, Jan Stolarek, Roly Perera, and James Cheney. 2017. Imperative Functional Programs That Explain Their Work. *Proceedings of the ACM on Programming Languages* 1, ICFP, Article 14 (2017), 28 pages. <https://doi.org/10.1145/3110258>
- Arvind Satyanarayan, Dominik Moritz, Kanit Wongsuphasawat, and Jeffrey Heer. 2017. Vega-Lite: A Grammar of Interactive Graphics. *IEEE Trans. Visualization & Comp. Graphics (Proc. InfoVis)* (2017).
- P. Griffiths Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price. 1979. Access Path Selection in a Relational Database Management System. In *Proceedings of the 1979 ACM SIGMOD International Conference on Management*

of Data (Boston, Massachusetts) (*SIGMOD '79*). Association for Computing Machinery, New York, NY, USA, 23–34. <https://doi.org/10.1145/582095.582099>

Ezekiel Soremekun, Lukas Kirschner, Marcel Böhme, and Andreas Zeller. 2021. Locating faults with program slicing: an empirical analysis. *Empirical Software Engineering* 26, 3 (01 Apr 2021), 51. <https://doi.org/10.1007/s10664-020-09931-7>

K. Truemper. 1989. On the delta-wye reduction for planar graphs. *Journal of Graph Theory* 13, 2 (1989), 141–148. <https://doi.org/10.1002/jgt.3190130202>